

Mini Focus Planner — Tarefa de Introdução a Testes de Front-end

Contexto

Você entregou uma primeira versão do **Mini Focus Planner** em **Next.js**.

A etapa anterior tinha como foco principal entender a estrutura de uma aplicação Next com App Router, componentização, estado no cliente, validação de formulário e persistência local.

Nesta nova etapa, o objetivo é introduzir **testes de front-end** dentro do projeto existente.

A proposta não é seguir uma receita pronta, copiar comandos e preencher arquivos mecanicamente. A proposta é aprender a transformar comportamentos da aplicação em verificações automatizadas.

Você deverá estudar, pesquisar, tomar decisões técnicas e justificar suas escolhas.

Objetivo da tarefa

Implementar uma primeira camada de testes automatizados para o projeto **sem usar Playwright neste momento**.

Nesta etapa, o foco será em:

- testes unitários;
- testes de componentes;
- testes de comportamento da interface;
- testes de store/estado;
- testes de validação;
- testes relacionados à persistência local;
- organização e documentação da estratégia de testes.

A entrega deve demonstrar que você entende:

- o que testar;
 - por que testar;
 - qual tipo de teste usar em cada caso;
 - como escrever testes legíveis;
 - como evitar testes frágeis;
 - como interpretar uma falha de teste.
-

Escopo técnico atual do projeto

O projeto atual possui uma estrutura próxima desta:

```
src/  
  app/  
    page.tsx  
  components/  
    addTaskForm.tsx  
    TaskList.tsx  
    TaskCard.tsx  
  hooks/  
    useStoreHydration.ts  
  store/  
    useTaskStore.ts  
  types/  
    index.ts
```

A aplicação usa:

- **Next.js**;
- **React**;

- **TypeScript**;
- **React Hook Form**;
- **Zod**;
- **Zustand**;
- **Zustand persist/localStorage**;
- **Tailwind CSS**.

Os testes devem partir desse código real, não de exemplos genéricos.

Stack esperada para esta etapa

Você deverá pesquisar e configurar uma stack de testes adequada para componentes React em um projeto Next.js.

Ferramentas esperadas:

- **Vitest**;
- **React Testing Library**;
- **Testing Library user-event**;
- **Testing Library jest-dom**;
- **jsdom**.

Nesta etapa, **não usar Playwright**.

A ideia é primeiro entender testes unitários e testes de componentes antes de avançar para testes end-to-end em navegador real.

O que você precisa estudar antes de implementar

Antes de escrever os testes, estude e anote respostas curtas para estas perguntas:

1. Qual é a diferença entre teste unitário, teste de componente e teste end-to-end?
2. Por que React Testing Library incentiva testar comportamento visível ao usuário?
3. Qual é a diferença entre `fireEvent` e `userEvent`?
4. O que é `jsdom` e por que ele é usado em testes de front-end?
5. Para que serve `@testing-library/jest-dom`?
6. Como testar componentes que dependem de Zustand?
7. Como isolar o estado da store entre testes?
8. Como testar código que usa `localStorage`?
9. O que torna um teste frágil?
10. Por que testar classes Tailwind diretamente normalmente é uma má ideia?

Essas respostas devem entrar em um arquivo de documentação no próprio projeto.

Mentalidade esperada

Testes devem verificar comportamento, não detalhes internos desnecessários.

Um bom teste responde perguntas como:

- O usuário consegue adicionar uma tarefa válida?
- O formulário impede uma tarefa sem título?
- A lista mostra o estado vazio corretamente?
- O filtro de pendentes mostra apenas tarefas pendentes?
- O filtro de concluídas mostra apenas tarefas concluídas?
- A paginação exibe apenas a quantidade esperada de tarefas?
- O botão de concluir altera o status da tarefa?
- O botão de excluir remove a tarefa?
- O modo de edição carrega os dados atuais?

- A store persiste os dados corretamente?

Um teste ruim tenta provar coisas como:

- o componente usa `useState` ;
- uma função interna tem determinado nome;
- uma classe Tailwind específica está presente sem relação clara com comportamento;
- o HTML interno continua exatamente igual após qualquer refatoração;
- uma implementação específica foi chamada sem necessidade.

Regra prática:

| *Teste aquilo que quebraria a experiência do usuário se parasse de funcionar.*

Áreas do projeto que devem orientar os testes

1. Store de tarefas

Arquivo principal:

```
src/store/useTaskStore.ts
```

A store é uma boa candidata para testes porque concentra regras importantes da aplicação:

- adicionar tarefa;
- alternar status;
- editar tarefa;
- excluir tarefa;
- manter formato consistente da entidade `Task` ;
- gerar `id` ;
- definir `createdAt` e `updatedAt` ;
- persistir em storage.

Você deve identificar quais comportamentos podem ser testados diretamente na store sem renderizar componentes.

Perguntas que seus testes devem ajudar a responder:

- Ao adicionar uma tarefa, ela entra na lista?
- Uma tarefa nova começa como `pending` ?
- A tarefa recebe `id` , `createdAt` e `updatedAt` ?
- Ao alternar status, `pending` vira `done` e `done` vira `pending` ?
- Ao editar uma tarefa, os dados são atualizados sem perder os campos antigos?
- Ao editar, `updatedAt` muda?
- Ao excluir, apenas a tarefa correta é removida?
- A store começa limpa em cada teste?

Atenção: como Zustand mantém estado em memória, você precisa pesquisar como resetar ou isolar a store entre testes.

2. Formulário de criação

Arquivo principal:

```
src/components/addTaskForm.tsx
```

O formulário usa **React Hook Form** e **Zod**.

Você deve testar o comportamento do formulário do ponto de vista do usuário.

Cenários esperados:

- formulário renderiza campo de título;
- formulário renderiza campo de descrição;

- formulário renderiza campo de categoria;
- formulário renderiza seleção de prioridade;
- prioridade padrão é coerente com a implementação atual;
- tentativa de envio sem título exibe erro de validação;
- envio com título válido adiciona uma tarefa;
- após envio válido, o formulário limpa os campos esperados.

Evite testar diretamente se `useForm` foi chamado. Isso é detalhe interno.

Teste o que aparece na tela e o efeito final da interação.

3. Listagem, filtros e paginação

Arquivo principal:

```
src/components/TaskList.tsx
```

Esse componente contém várias responsabilidades importantes:

- carregar tarefas da store;
- lidar com hidratação;
- filtrar tarefas por status;
- paginar resultados;
- renderizar estado vazio;
- trocar página;
- resetar página ao mudar filtro.

Cenários esperados:

- mostra mensagem de carregamento antes da hidratação;
- mostra estado vazio quando não há tarefas;
- mostra tarefas existentes;
- filtro "Todas" mostra tarefas pendentes e concluídas;
- filtro "Pendentes" mostra apenas tarefas pendentes;
- filtro "Concluídas" mostra apenas tarefas concluídas;
- paginação aparece quando houver mais itens do que o limite por página;
- botão "Próxima" muda para a próxima página;
- botão "Anterior" retorna para a página anterior;
- troca de filtro reseta para a primeira página.

Ponto de atenção técnico:

No código atual, existe uma atualização de estado durante a renderização quando `currentPage > totalPages`.

Você deve pesquisar por que atualizar estado diretamente durante renderização pode ser problemático em React e pensar se isso deveria ser refatorado antes ou durante a escrita dos testes.

A tarefa não é apenas "fazer o teste passar". A tarefa também é perceber quando o teste expõe um problema de design do componente.

4. Card da tarefa

Arquivo principal:

```
src/components/TaskCard.tsx
```

Esse componente concentra interações diretas com uma tarefa:

- exibir título;
- exibir descrição quando existir;

- exibir prioridade;
- marcar como concluída;
- editar;
- cancelar edição;
- salvar edição;
- excluir.

Cenários esperados:

- renderiza título da tarefa;
- renderiza descrição quando informada;
- não renderiza descrição vazia como conteúdo quebrado;
- mostra prioridade traduzida para o usuário;
- checkbox reflete status atual;
- clicar no checkbox alterna status;
- clicar em editar troca para modo de edição;
- modo de edição carrega os dados atuais da tarefa;
- cancelar edição volta para visualização normal;
- salvar edição atualiza a tarefa;
- excluir remove a tarefa.

Ponto de atenção:

Botões com emoji ou apenas `title` podem ser menos acessíveis para testes e para usuários. Se os testes tiverem dificuldade para encontrar esses botões de forma semântica, avalie melhorar a acessibilidade com `aria-label` ou texto acessível.

Essa melhoria faz parte da maturidade do teste: teste bom frequentemente denuncia interface pouco acessível.

5. Hook de hidratação

Arquivo principal:

```
src/hooks/useStoreHydration.ts
```

Esse hook existe para evitar problemas de hidratação ao usar Zustand persistido no localStorage.

Você deve pesquisar:

- o que é hidratação em Next.js/React;
- por que `localStorage` só existe no cliente;
- por que componentes client-side ainda podem ter cuidado com estado inicial;
- como testar um hook;
- quando vale a pena testar um hook isoladamente e quando é melhor testar o comportamento dele através de um componente.

Não é obrigatório criar um teste isolado para esse hook se você justificar bem por que ele está sendo coberto indiretamente pela `TaskList`.

Mas você precisa entender o problema que ele tenta resolver.

Entregáveis obrigatórios

1. Configuração de testes

Adicionar configuração funcional para rodar testes com Vitest e React Testing Library.

Você deve pesquisar a configuração adequada para projeto Next.js com TypeScript e alias de importação.

A entrega deve incluir scripts no `package.json` para:

- rodar testes em modo watch;
- rodar testes uma vez;

- gerar relatório de coverage, se configurado.

Os nomes dos scripts ficam a seu critério, desde que estejam documentados.

2. Testes da store

Criar testes para os comportamentos principais de `useTaskStore`.

Mínimo esperado:

- adicionar tarefa;
- alternar status;
- editar tarefa;
- excluir tarefa;
- garantir isolamento de estado entre testes.

Critério importante:

Os testes não devem depender da ordem de execução uns dos outros.

3. Testes do formulário

Criar testes para `TaskForm`.

Mínimo esperado:

- renderização dos campos principais;
 - validação de título obrigatório;
 - criação de tarefa válida;
 - limpeza do formulário após criação.
-

4. Testes da lista

Criar testes para `TaskList`.

Mínimo esperado:

- estado vazio;
 - renderização de tarefas;
 - filtro por status;
 - paginação básica.
-

5. Testes do card

Criar testes para `TaskCard`.

Mínimo esperado:

- renderização dos dados da tarefa;
 - alteração de status;
 - modo de edição;
 - exclusão.
-

6. Documentação curta

Criar arquivo:

```
docs/testing-strategy.md
```

O arquivo deve explicar:

1. quais ferramentas foram usadas;
2. por que essas ferramentas foram escolhidas;
3. quais tipos de comportamento foram testados;
4. quais partes do projeto ficaram fora desta etapa;
5. como rodar os testes;
6. quais dificuldades apareceram;
7. quais melhorias no código foram percebidas por causa dos testes.

Não precisa ser longo. Precisa ser claro.

Critérios de aceite

A entrega será considerada satisfatória se:

- o projeto instala as dependências sem conflito evidente;
 - os testes rodam por script documentado;
 - os testes passam localmente;
 - os testes são legíveis;
 - cada teste tem nome descritivo;
 - há isolamento entre testes;
 - a store não vaza estado de um teste para outro;
 - os testes verificam comportamento observável;
 - validação de formulário é coberta;
 - filtros são cobertos;
 - paginação é coberta;
 - ações do card são cobertas;
 - a documentação explica decisões técnicas;
 - eventuais refatorações são justificadas.
-

Critérios de avaliação

1. Entendimento

Você consegue explicar o que cada teste garante?

Se um teste falhar, você consegue explicar qual comportamento quebrou?

2. Qualidade dos testes

Os testes protegem fluxos importantes ou apenas aumentam número de coverage?

3. Legibilidade

Os testes são fáceis de ler?

O nome do teste descreve o comportamento?

4. Independência

Um teste depende do outro?

A store é resetada entre cenários?

O localStorage é limpo quando necessário?

5. Acessibilidade

Os componentes são fáceis de testar usando queries como `getByRole`, `getByLabelText` e `getByText`?

Se não forem, o que isso revela sobre a interface?

6. Capacidade de refatoração

Os testes permitem melhorar a implementação sem quebrar por detalhes irrelevantes?

Ou eles estão presos demais à estrutura interna do componente?

Restrições da tarefa

Nesta etapa, não implementar:

- Playwright;
- Cypress;
- testes end-to-end reais;
- CI/CD;
- snapshot testing como estratégia principal;
- testes pixel-perfect;
- testes focados em classes Tailwind isoladas;
- mocks complexos desnecessários.

A prioridade é aprender bem a base antes de avançar.

Sugestão de caminho de trabalho

Esta seção não é um passo a passo fechado. Use como orientação de raciocínio.

Primeiro

Entenda o projeto atual e liste os comportamentos críticos.

Não comece instalando biblioteca sem saber o que quer testar.

Depois

Pesquise como Vitest e React Testing Library são configurados em um projeto Next.js com TypeScript.

Compare pelo menos duas fontes: documentação oficial e um exemplo prático confiável.

Em seguida

Comece pelos testes mais previsíveis: store e regras puras.

Eles tendem a ser mais fáceis de debugar.

Depois avance para componentes

Teste o formulário, a lista e o card.

Durante esse processo, observe se a dificuldade de teste está apontando para algum problema real do código.

Por fim

Documente o que foi feito e revise se os testes realmente protegem o MVP.

Perguntas que devem guiar sua implementação

Antes de criar cada teste, complete mentalmente a frase:

| *Este teste garante que o usuário consegue...*

Ou:

| *Este teste garante que a regra de negócio...*

Se você não conseguir completar uma dessas frases, provavelmente o teste está mal definido.

Resultado esperado

Ao final desta tarefa, o projeto deve ter uma base inicial de testes que ajude a validar os principais comportamentos do Mini Focus Planner.

Mais importante do que a quantidade de testes é a qualidade da leitura, a coerência dos cenários e a capacidade de explicar as decisões tomadas.

A entrega ideal não é uma suíte enorme.

A entrega ideal é uma primeira suíte pequena, bem pensada, que mostre que você entendeu como testar uma aplicação React/Next real.